

Project Documentation

Date	29 September 2026
Team ID	SWTID-2026-3833
Project Name	AI StudyBuddy API
Maximum Marks	20 Marks

1. Team Details

S.No	Name	Role	Email
1	Bharathi V	Team Lead	bharathiv714@gmail.com
2	Lereina K A	Member	lereinaka@gmail.com
3	Mageshwaran S	Member	mageshwaran03037@gmail.com
4	Yoga Prasanna J	Member	yp2232006@gmail.com
5	Deepa Tracy Wins C	Member	tracydeepa@gmail.com

Team ID	SWTID-2026-3833
Project Title	AI StudyBuddy – AI-Powered Learning Assistance & Study Support Backend
GitHub Repository	https://github.com/bharathi1111/Studybuddy_teamProject
Demo Video	Demo recording (Google Drive)

2. Project Description

AI StudyBuddy is an AI-powered learning assistance platform that simplifies the study process for students by combining a secure Node.js/Express/MongoDB Atlas backend with the Google Gemini generative AI model. Authenticated students upload study material and receive AI-generated summaries, flashcards, multiple-choice quizzes and personalized study plans, while administrators manage registered users and monitor system usage. JWT-based authentication (with access and refresh tokens) and bcryptjs password hashing secure every account, and Role-Based Access Control ensures only authorized users reach premium AI-powered features.

3. Scenario-Based Case Study

Persona: Rahul, a second-year engineering student preparing for multiple semester examinations while also learning new technical skills for internships and placement opportunities. His study material is scattered across lecture notes, PDFs, textbooks, online articles and handwritten notes, making it difficult to organize and revise effectively within his limited preparation time. Traditional study methods require significant manual effort to summarize material, prepare practice questions and build a revision schedule – a process that is repetitive, time-consuming and often inconsistent. After adopting AI StudyBuddy, Rahul uploads his material once, and Google Gemini generates a summary, a set of flashcards, a self-assessment quiz and a study plan tailored to his exam timeline – work that used to consume an entire evening now takes a few API calls.

4. Software & Hardware Requirements

Category	Requirement
Operating System	Windows 10/11, macOS, or Linux (cross-platform)
Runtime	Node.js v16 or above
Package Manager	npm v8 or above
Framework	Express.js (RESTful routing)
Database	MongoDB Atlas (managed cloud cluster) with Mongoose ODM
AI Service	Google Gemini API (@google/generative-ai , model gemini-2.5-flash)
Development Tools	Visual Studio Code, Postman / Thunder Client, Git & GitHub
Minimum Hardware	Intel Core i5 (8th Gen+) / AMD Ryzen 5 or equivalent, 8 GB RAM (16 GB recommended), 1 GB available disk space

EPIC 1 - System Architecture

AI STUDY BUDDY – PROJECT ARCHITECTURE

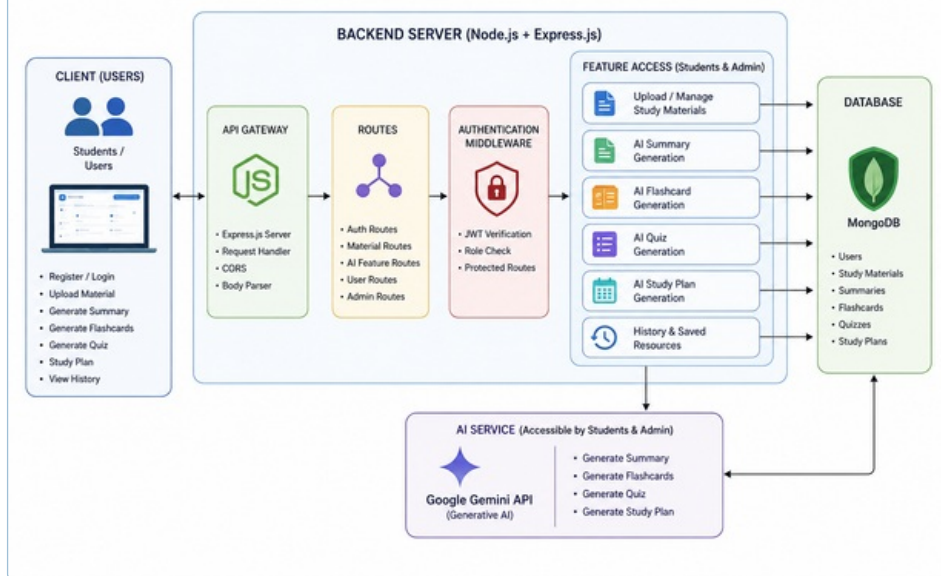


Fig. 1 – Client (React) → API Gateway (Express.js) → Routes → Authentication Middleware → Feature Controllers → MongoDB, with a parallel AI Service path to Google Gemini

ER Diagram - Data Model

Currently implemented model: two collections – **User** (_id, name, email [unique, required], password [bcryptjs-hashed], role [Student/Admin], createdAt) and **Material** (_id, user [reference to User], title, content [raw uploaded text], filename, summary, flashcards [array of {question, answer}], quiz [array of {question, options[], answer}], studyPlan, timestamps). Each AI-generation call (summarize / flashcards / quiz / study-plan) writes its result directly onto the originating Material document rather than into a separate collection.

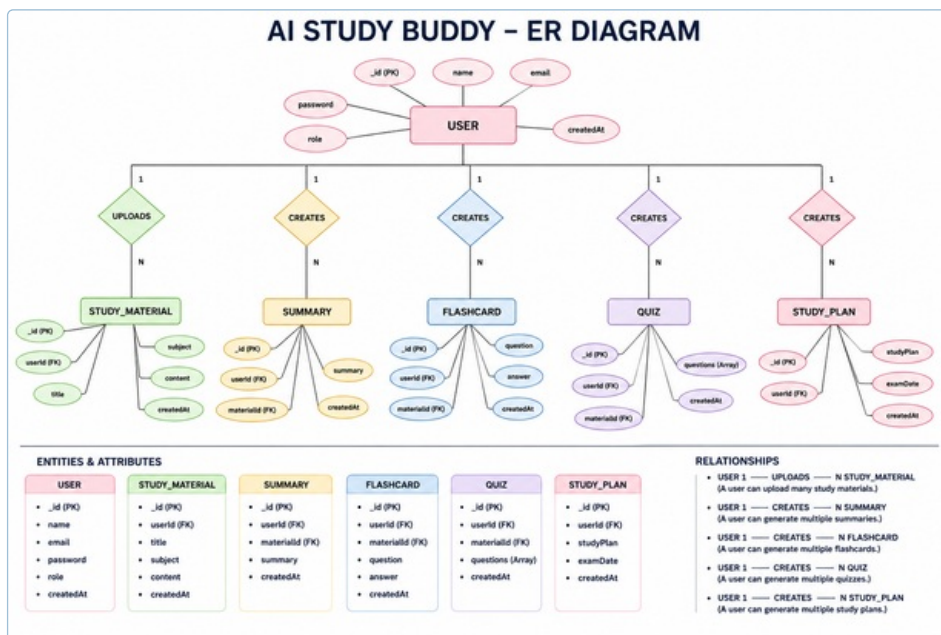


Fig. 2 – Full ER diagram as originally designed by the team, showing Summary, Flashcard, Quiz and Study Plan as separate normalized collections linked to Study Material. The team implemented a denormalized version of this design in this cycle – all four AI outputs live as fields directly on the Material document – with the fully separated, independently-queryable collections shown here recorded as the normalization target for the Scalability & Future Plan roadmap (see Phase 8).

Key Features

- Secure JWT-based authentication (access + refresh tokens) with bcryptjs password hashing
- Role-Based Access Control distinguishing Student and Administrator privileges
- Study material upload with type/size-validated storage (multer)
- Google Gemini-powered AI summary generation
- Google Gemini-powered AI flashcard generation
- Google Gemini-powered AI quiz generation
- Google Gemini-powered AI personalized study plan generation
- Administrative user and system management
- Centralized error handling with structured JSON responses

Roles and Responsibilities

Role	Capabilities
Student	Register, log in, upload study materials, generate AI summaries/flashcards/quizzes/study plans, and view/manage previously generated learning resources.
Administrator	Manage registered user accounts, monitor application and AI-service usage, review system activity, and ensure smooth backend operations.

User Flow

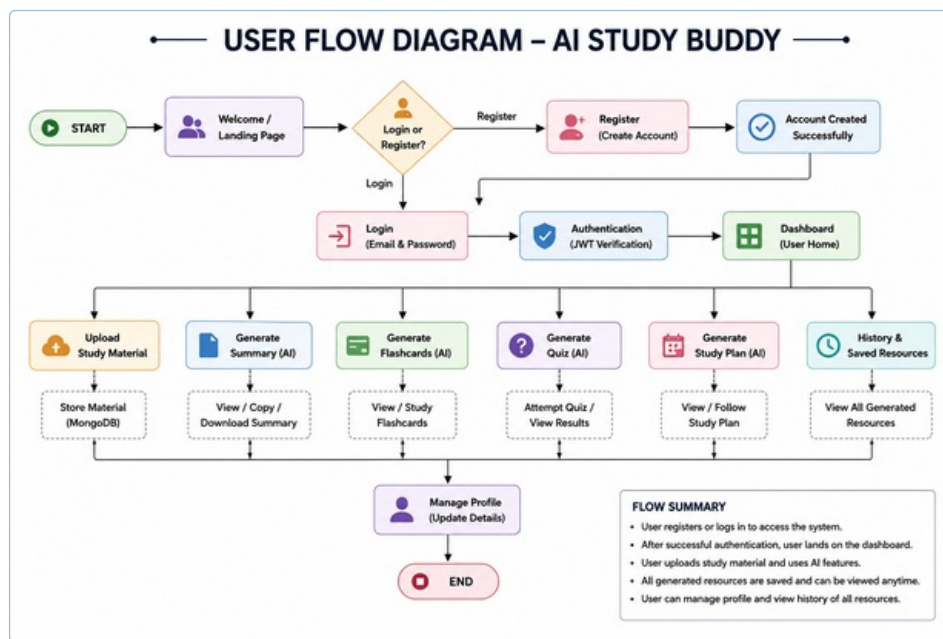


Fig. 3 – Register/Login → JWT Authentication → Dashboard → Upload Study Material → Generate Summary/Flashcards/Quiz/Study Plan → View/Save Resources → Manage Profile

MVC Pattern

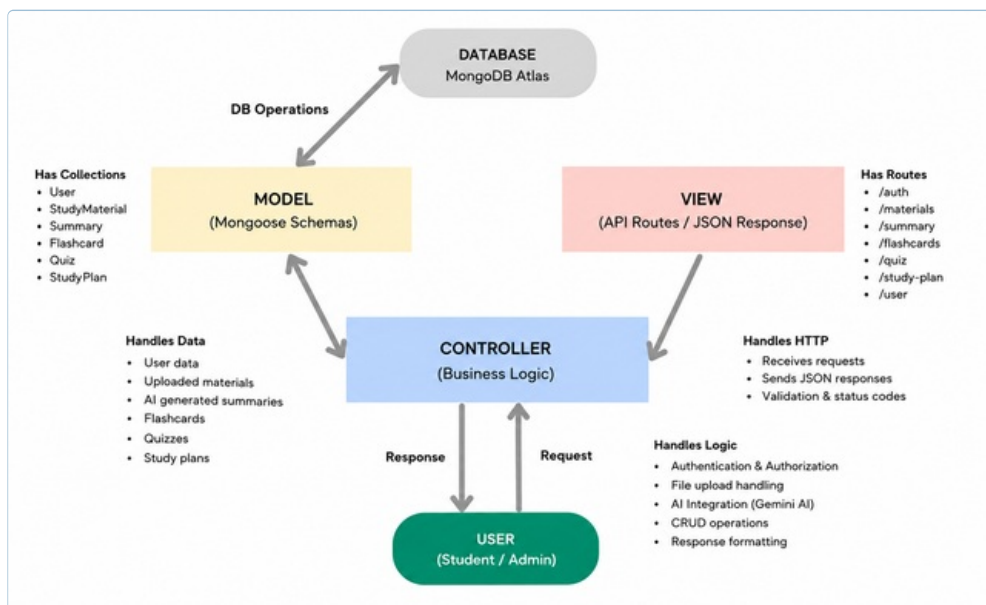


Fig. 4 – Model (Mongoose schemas over MongoDB Atlas) holds data and validation; Controller handles auth, file upload and Gemini AI integration; the JSON response itself serves as the "View" for this API-only project.

EPIC 2 - Project Setup

Created the `AI-StudyBuddy` project folder, initialized it with `npm init -y`, and installed `express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai` plus `nodemon` as a dev dependency. Scaffolded the `src/` folder (controllers, middleware, models, routes, utils) alongside `index.js`, `.env`, `package.json` and `.gitignore`. Configured `.env` with `PORT=5000`, `MONGO_URI`, JWT secrets and `GEMINI_API_KEY`.

EPIC 3 - Backend Development

The backend is built around five core scripts. `index.js` is the application entry point: it loads environment variables, applies `cors` and JSON body-parsing, mounts the `/api/auth`, `/api/materials` and `/api/admin` routers, exposes a `GET /health` check, and only starts listening on port 5000 once `connectDB()` resolves. `src/utils/db.js` establishes the Mongoose connection to MongoDB Atlas. `src/middleware/auth.js` exports `protect` (Bearer-token verification via `JWT_ACCESS_SECRET`) and `adminOnly` (role check). `src/utils/gemini.js` wraps the Google Gemini `gemini-2.5-flash` model behind a single `askGemini(prompt)` function. `src/middleware/upload.js` configures `multer` disk storage with a unique filename per upload and a file filter restricted to `.txt` / `.md` / `.pdf` under 5 MB.

Backend Development Demo: [AI_Studybuddy_Backend_Demo.mkv](#)

EPIC 4 - Database Configuration

MongoDB Atlas is the primary NoSQL datastore, accessed through Mongoose for schema validation and structured querying. The **User** schema (`models/User.js`) stores `name`, a required and unique `email`, a `bcryptjs`-hashed `password`, and a `role` of `Student` or `Admin`. The **Material** schema below is where all study content and AI-generated output actually live in this cycle.

```
src > models > JS Material.js > ...
1  const mongoose = require("mongoose");
2
3  const materialSchema = new mongoose.Schema(
4  {
5      user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
6      title: { type: String, required: true },
7      content: { type: String, required: true }, // raw text content
8      filename: { type: String }, // original uploaded file name
9      summary: { type: String },
10     flashcards: [
11         {
12             question: String,
13             answer: String,
14         },
15     ],
16     quiz: [
17         {
18             question: String,
19             options: [String],
20             answer: String,
21         },
22     ],
23     studyPlan: { type: String },
24 },
25 { timestamps: true }
26 );
27
28 module.exports = mongoose.model("Material", materialSchema);
```

models/Material.js – a User reference plus title/content/filename, and the four AI outputs (summary, flashcards array, quiz array, studyPlan) embedded directly on the same document, with timestamps: true

Database Collections

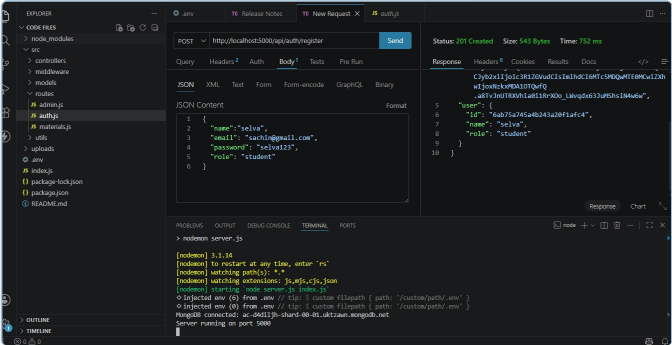
Collection	Stores
Users	Account and authentication details for Students and Administrators
Materials	Uploaded study content plus every AI-generated summary, flashcard set, quiz and study plan derived from it

EPIC 5 - Execution & API Testing Evidence

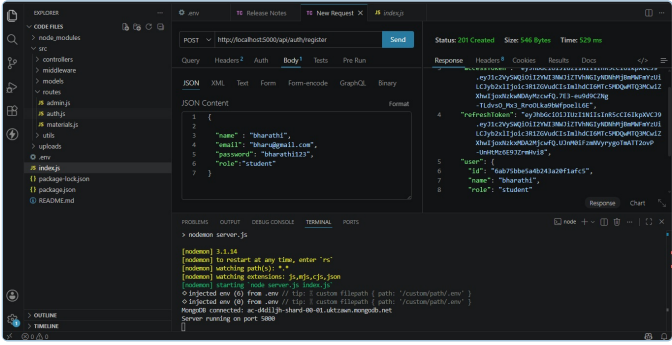
API Route Interface Endpoints

Group	Endpoint	Method	Purpose
Authentication	/api/auth/register	POST	Registers a new user and returns access + refresh tokens
	/api/auth/login	POST	Authenticates a registered user and returns fresh tokens
	/api/auth/logout	POST	Ends the current session
Study Material	/api/materials/upload	POST	Uploads and stores study material for AI processing
	/api/materials/:id	GET/PUT	Retrieves or updates a stored material
AI Features	/api/materials/:id/summarize	POST	Generates an AI summary
	/api/materials/:id/flashcards	POST	Generates AI flashcards
	/api/materials/:id/quiz	POST	Generates an AI quiz
	/api/materials/:id/study-plan	POST	Generates an AI study plan
Administrative	/api/admin/*	GET/PUT/DELETE	Manages users and monitors system activity

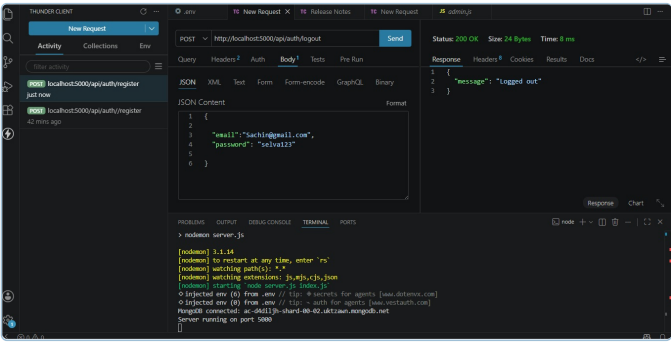
Test Evidence



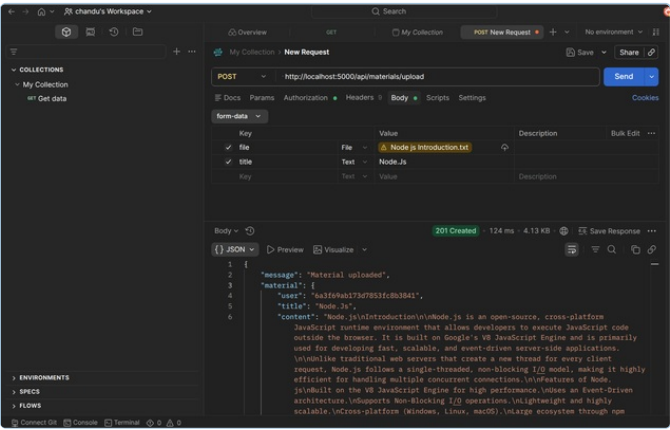
Register (user "selva") – 201 Created, 752 ms, access + refresh tokens issued



Register (user "bharathi") – 201 Created, 529 ms, MongoDB Atlas connection visible in terminal



Logout – 200 OK, 8 ms



Upload Study Material – 201 Created, 124 ms

Demo & Repository

Resource	Link
GitHub Repository	https://github.com/bharathi1111/Studybuddy_teamProject
Backend Development Demo	AI_Studybuddy_Backend_Demo.mkv
Final Demo Video	Google Drive demo recording

Conclusion

AI StudyBuddy delivers a complete, secure, AI-augmented learning-assistance backend within the 15-29 September 2026 cycle: JWT-and-RBAC-protected study material management backed by MongoDB Atlas, and four Google Gemini-powered endpoints that turn a single uploaded file into a summary, flashcards, a quiz and a personalized study plan – directly resolving the scattered, manual, and guidance-less revision experience identified in the team's initial problem statements.